

CTRX8188F - Cascading

About this document

Scope and purpose

In this application note a guidance on how to configure Infineon's CTRX8188F as PRIMARY and SECONDARY MMIC, as used in a cascaded system architecture, is given. Exemplary, a system consisting out of four MMICs is considered. Information provided does not replace information given in the User Manual [1].

Note: Please note that the description given in this application note is only valid for RAM FW versions approved by Infineon accordingly as defined in the latest safety package release note [4] available via MyICP.

Note: This is a DRAFT version and still subject to change.

Intended audience

This document is intended for all engineers working with Infineon's Radar MMIC CTRX8188F.

Related devices

Product type	Package	Marking
CTRX8188F	PG-TF2BGA-457-01	CTRX8188F

CTRX8188F - Cascading

Table of contents

About this document.....	1
Table of contents.....	2
1 Cascading of MMIC.....	3
1.1 Local oscillator distribution	4
1.2 Clock distribution	5
1.3 Synchronization	8
1.4 RX analog frontend compensation	8
1.4.1 Amplitude response	8
1.4.2 Phase response	9
1.4.3 Implementation	10
2 Firmware configuration flow	11
2.1 50 MHz clock distribution.....	12
2.2 Providing clock source to peripheral (optional, only for PRIMARY MMIC)	12
2.3 LO distribution.....	13
2.4 Calibration	14
2.5 Sequencing	15
2.6 Monitoring	15
2.7 Pseudocode	16
3 Terminology	25
4 References	27
5 Revision history	28

CTRX8188F - Cascading

1 Cascading of MMIC

Generally, higher angular resolution in azimuth and elevation is the main reason for using multiple radar MMICs in a cascaded setup within an automotive radar sensor system. In addition to an increased channel count, and therefore an increased antenna aperture, an improved measurement capability in both azimuth and elevation can be realized.

In this chapter, an overview of a radar sensor system considered in this application note is given. In this context, a design example of a clock distribution and a local oscillator (LO) distribution network is presented. Furthermore, the chip configuration for cascading and synchronous triggering of the cascaded setup is shown.

Infineon's CTRX8188F, subsequently referred as MMIC, can be used as STAND-ALONE, PRIMARY or SECONDARY MMIC. Here, the focus is on a cascaded system architecture as depicted in [Figure 1](#), and therefore on the usage of the MMIC as PRIMARY and SECONDARY MMIC.

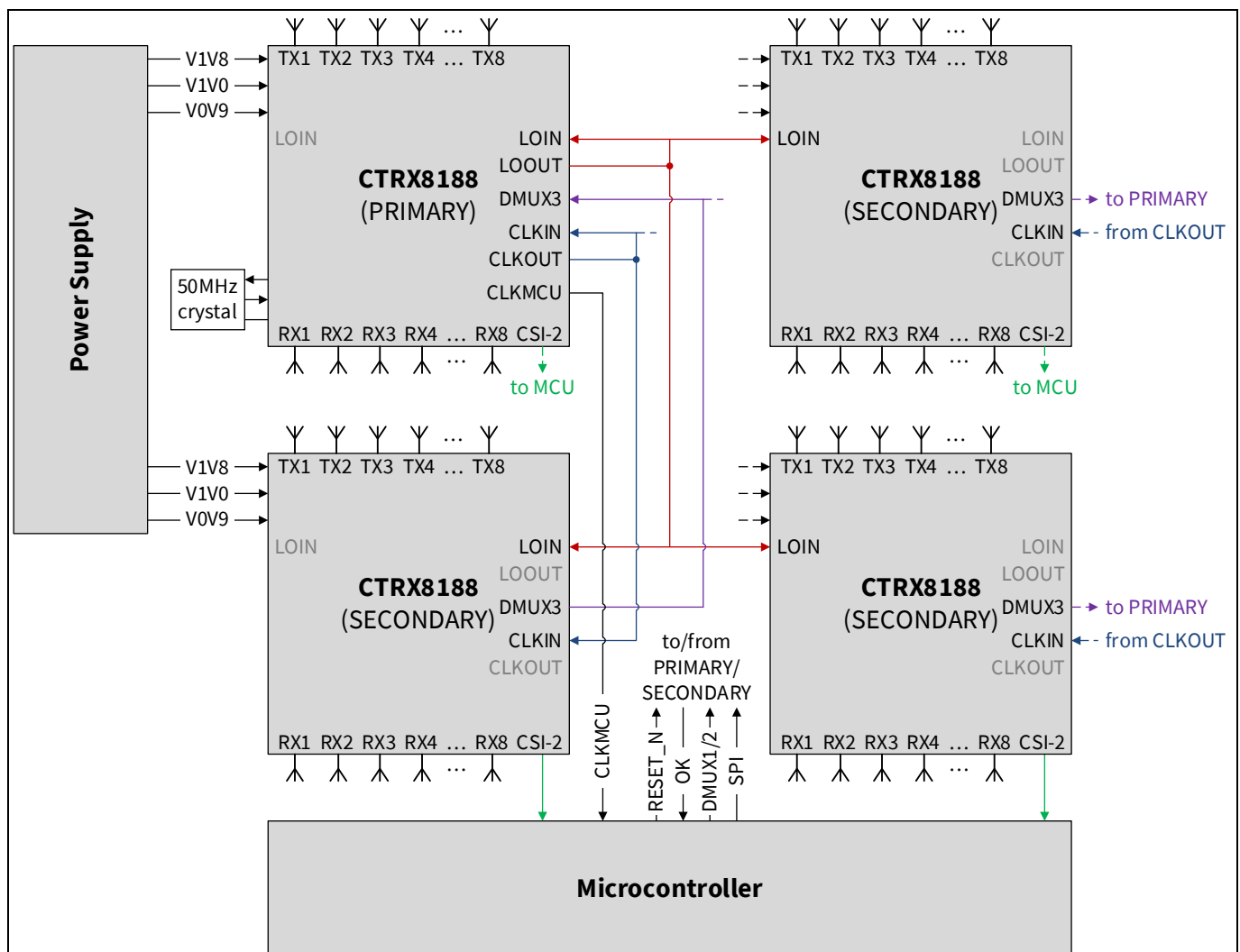


Figure 1: Cascading-based system architecture of a radar sensor (application example)

CTRX8188F - Cascading

By default, after powering-up or performing a hard reset, the MMIC is configured as STAND-ALONE MMIC.

- **STAND-ALONE MMIC**

In STAND-ALONE MMIC configuration a 50 MHz crystal or an external active clock source via AC coupling is used. Both, the 50 MHz clock and the LO signal are generated internally. No MMIC infrastructure for cascading such as CLKIN / CLKOUT and LOIN / LOOUT port is used.

- **PRIMARY MMIC**

In PRIMARY MMIC configuration an external crystal is used. Both, the 50 MHz clock and the LO signal are generated internally. The clock signal is available at CLKOUT and the LO signal is available at LOOUT. Each, CLKOUT and LOOUT signal is distributed to the CLKIN and LOIN (LOIN1 or LOIN2) of SECONDARY MMICs as well as PRIMARY MMIC (self-feeding). DMUX3 is used for synchronization with SECONDARY MMICs.

- **SECONDARY MMIC**

In SECONDARY MMIC configuration no external crystal is used. Both, the 50 MHz clock signal and the LO signal are provided at CLKIN and LOIN (LOIN1 or LOIN2) input. DMUX3 is used for synchronization with PRIMARY MMIC.

The system integrator configures each MMIC correspondingly to the usage in the system.

1.1 Local oscillator distribution

The LO signal is provided by the PRIMARY MMIC. Therefore, a network to distribute the LO signal to each MMIC is needed.

The system integrator has to ensure that the LO distribution network is

- a. impedance-wise matched to ensure appropriate signal quality,
- b. length-wise matched to ensure appropriate signal coherency, and
- c. designed to fulfill MMICs LO input power and phase noise specification.

In [Figure 2](#) the LO distribution network designed for a sensor system as initially introduced is shown.

CTRX8188F - Cascading

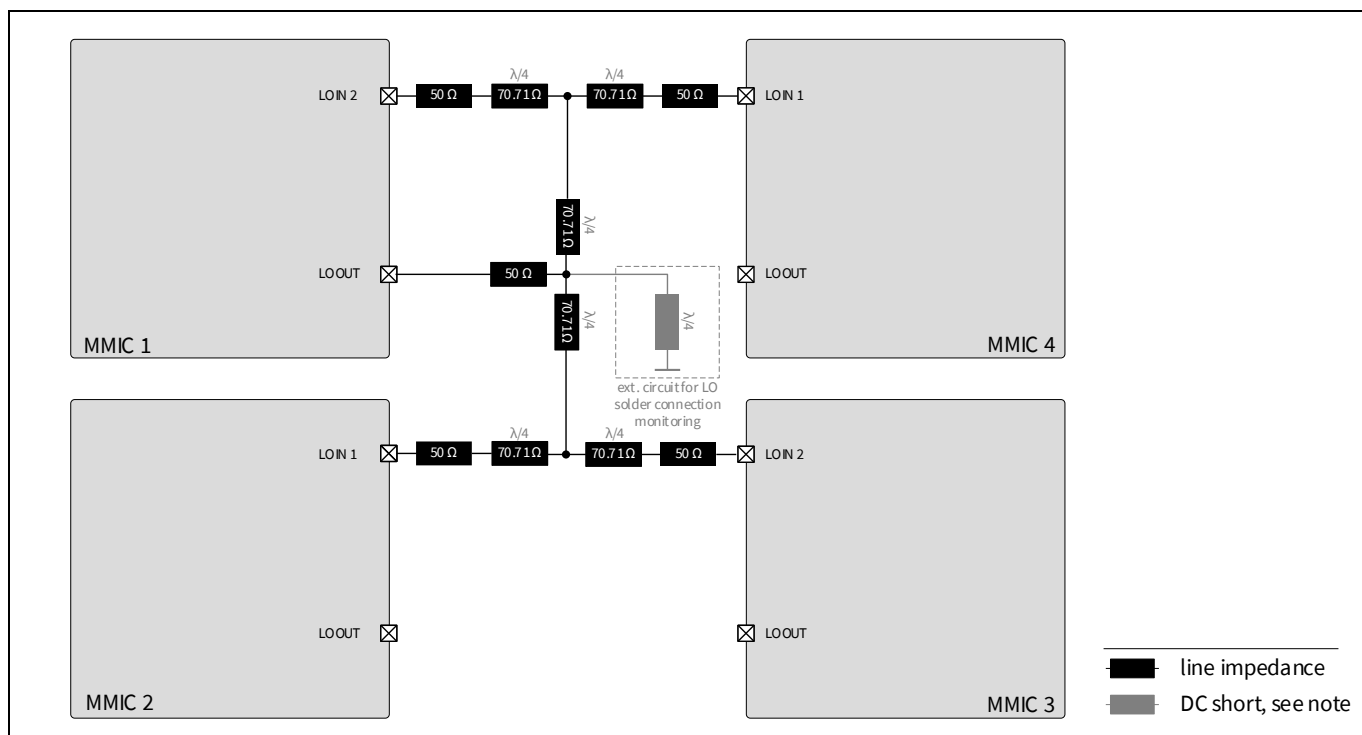


Figure 2: LO distribution network (application example)

Note: LO solder connection monitoring requires an external DC connection from each LO ball to GND on the PCB. A detailed description on the safety mechanism is given in the Safety Manual [].

1.2 Clock distribution

In addition to the LO signal, the distribution of the clock signal is required. The clock signal operates at a frequency of 50 MHz and is supplied via the CLKOUT pin of the PRIMARY MMIC. It must be fed to the CLKIN pins of both the SECONDARY MMICs and the PRIMARY MMIC itself. Similar to the LO distribution network, the system integrator ensures that the line impedances are stable and well-defined, and that the line lengths to each device are uniform in order to achieve a synchronous clock at the CLKIN pins.

The CLKOUT source resistance R_{scr} is selectable between 40 Ω and 50 Ω by executing the firmware command `Configure_MMIC_Clock()`. To achieve enough signal amplitude at the clock inputs it is recommended to use the 40 Ω setting in case of cascading four MMICs. The clock output signal should be split into the required number of clock input pins by a resistive splitter with resistance values designed in a way to match the characteristic line impedance from the clock output while at the same time matching the line impedance towards the clock inputs.

In [Figure 3](#), [Figure 4](#) and [Figure 5](#) the clock distribution schematic including the splitting circuit for different configurations is shown; two, three and four MMIC system.

Note: The CLKOUT source resistor R_{scr} is selectable; 50 Ω or 40 Ω. A detailed description is given in the User Manual [1].

The resistors of the clock splitter should be placed as close as possible to each other on the PCB while maintaining a symmetric layout. [Table 1](#) shows the recommended settings and resistor values for 2, 3 and 4 MMIC configurations.

CTRX8188F - Cascading

Table 1: Recommended PRIMARY MMIC CLKOUT configuration, splitter resistances for a 2, 3 and 4 MMIC configuration (resistor component selection from E48 $\pm 1\%$ tolerance series).

n	Rscr	Z1	Z2	Ra	Rb	Rc
2	50 Ω	50 Ω	50 Ω	150 Ω (147R2)	75.0 Ω (75R0)	75.0 Ω (75R0)
3	50 Ω	50 Ω	50 Ω	-	86.6 Ω (86R6)	63.4 Ω (64R9)
4	40 Ω	40 Ω	50 Ω	-	98.0 Ω (100R)	62.0 Ω (61R9)

Note: Please do not hesitate to contact the Infineon support in case of using a different system configuration, e.g. $n > 4$ or different characteristic line impedances.

To maintain the high signal integrity provided by the CLKOUT pin through the distribution network up to the CLKIN pins additional care must be taken by the system integrator. Other signal lines, especially with fast switching digital signals, should not be routed close to the clock line. Shielding the clock line with the common ground on the PCB or the CLKOUT_REF and CLKIN_REF lines can be considered (to be connected to one net and to be connected to the common ground ideally close to the CLKOUT_REF pin). For EMC and EMI reasons it is also advisable to embed the clock line in an inner layer of the PCB in-between ground planes.

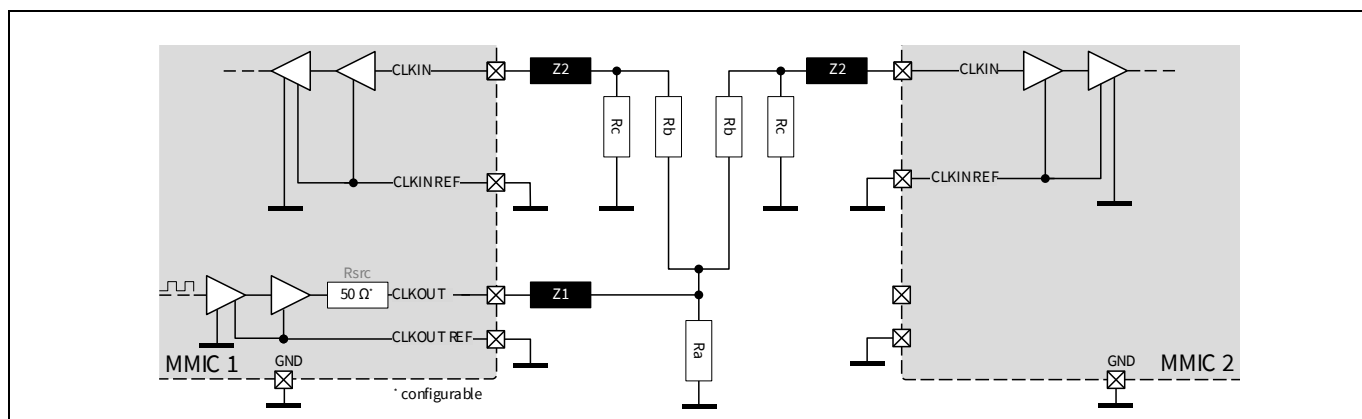


Figure 3: Clock distribution network, 2 MMICs (application example)

CTRX8188F - Cascading

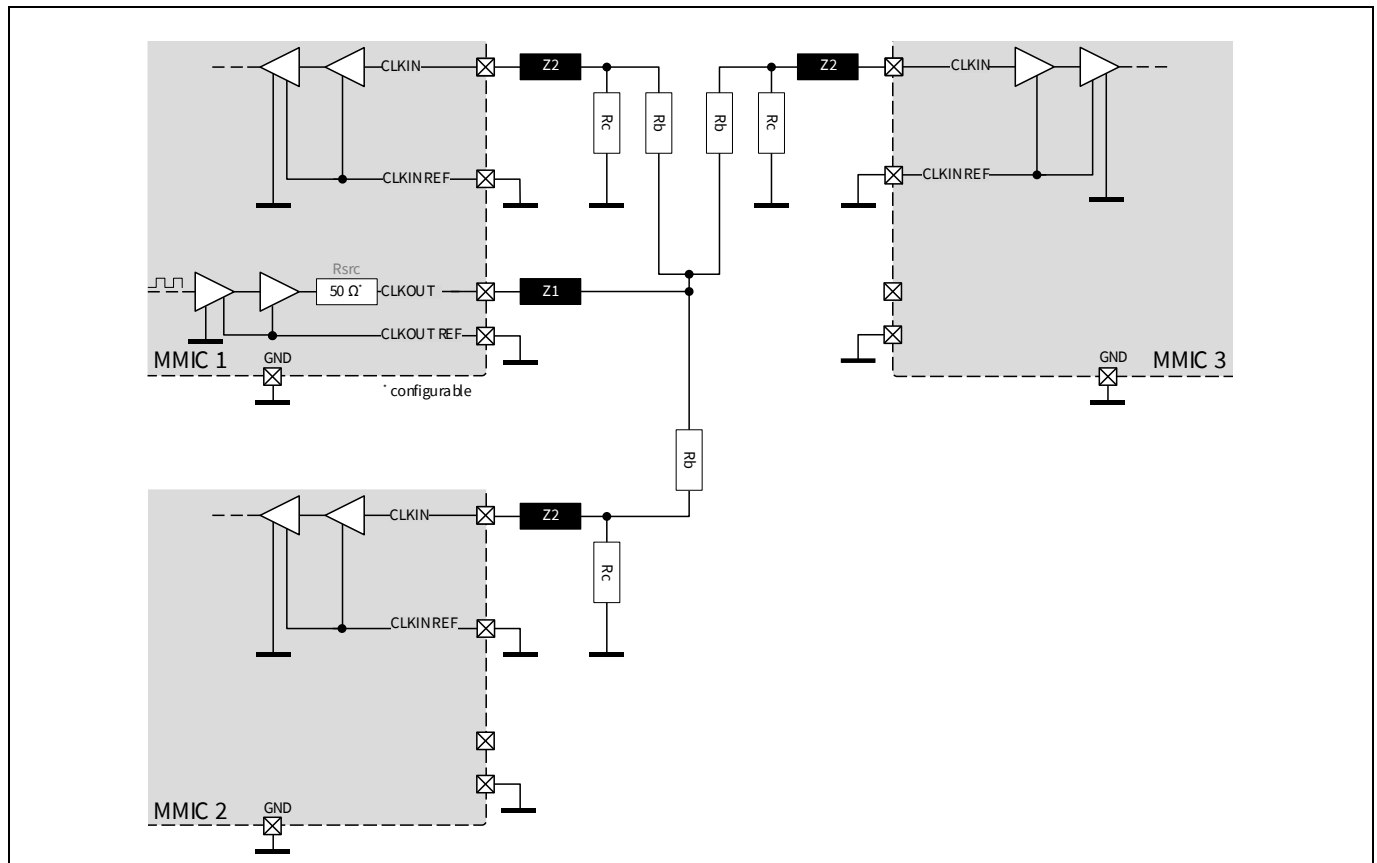


Figure 4: Clock distribution network, 3 MMICs (application example)

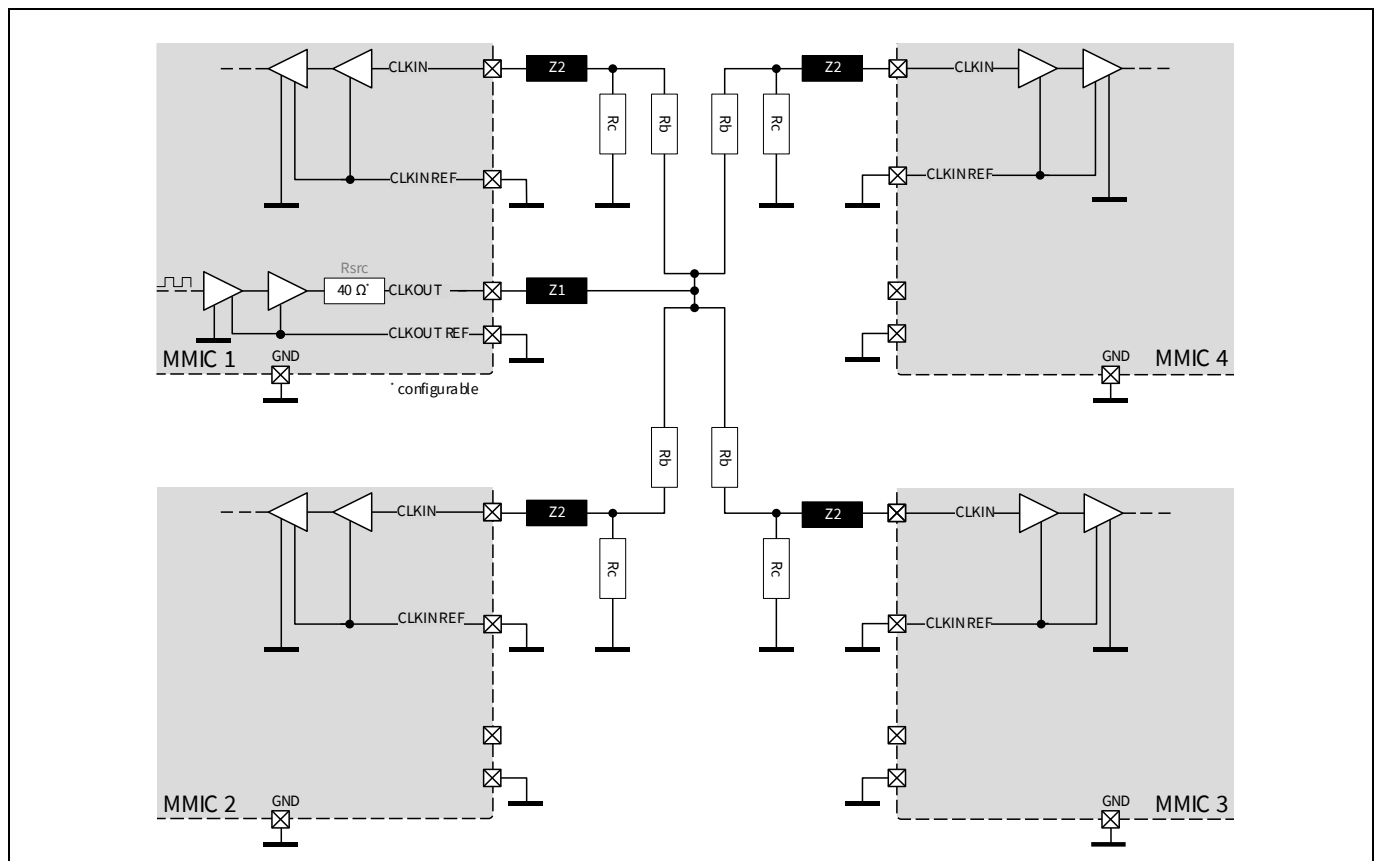


Figure 5: Clock distribution network, 4 MMICs (application example)

CTRX8188F - Cascading

1.3 Synchronization

For a time-wise synchronization between the MMICs during operation there is a trigger signal modulated on top of the 50 MHz clock signal as depicted in [Figure 6](#). The trigger event (= missing pulse) is provided by PRIMARY MMIC. Additionally, the DMUX3 is connected between the MMICs for synchronization of calibration and monitoring routines.

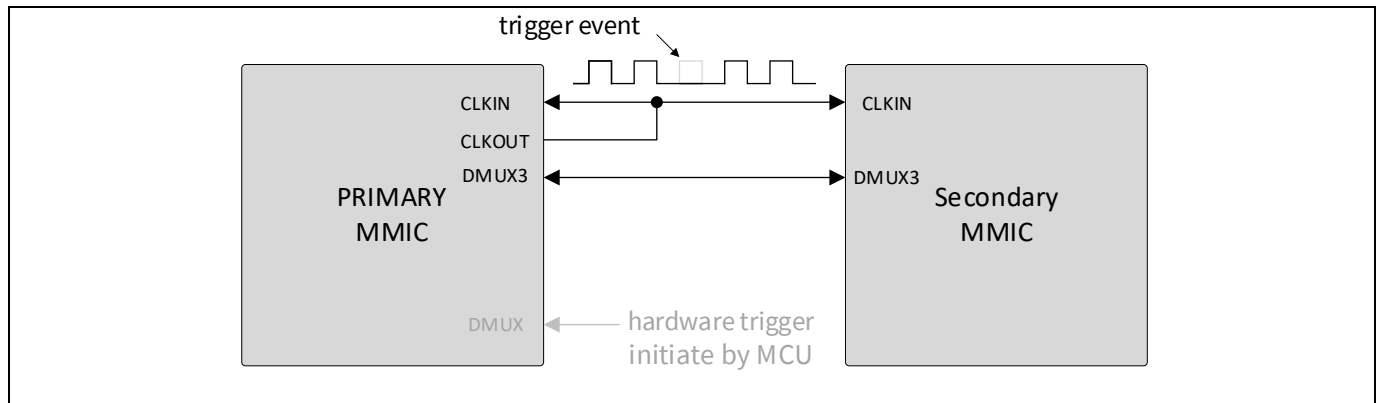


Figure 6: Clock-based synchronization

1.4 RX analog frontend compensation

This chapter describes a compensation method for manufacturing process variations in the receiver analog frontend between two or more MMIC within a cascaded system.

Note: Please note that the method including the model description is preliminary and the might be changed after device verification.

The phase response of several devices before compensation (left side) and after compensation (right side) is shown in [Figure 7](#). The process variation of the phase response can be improved significantly by applying the following compensation approach.

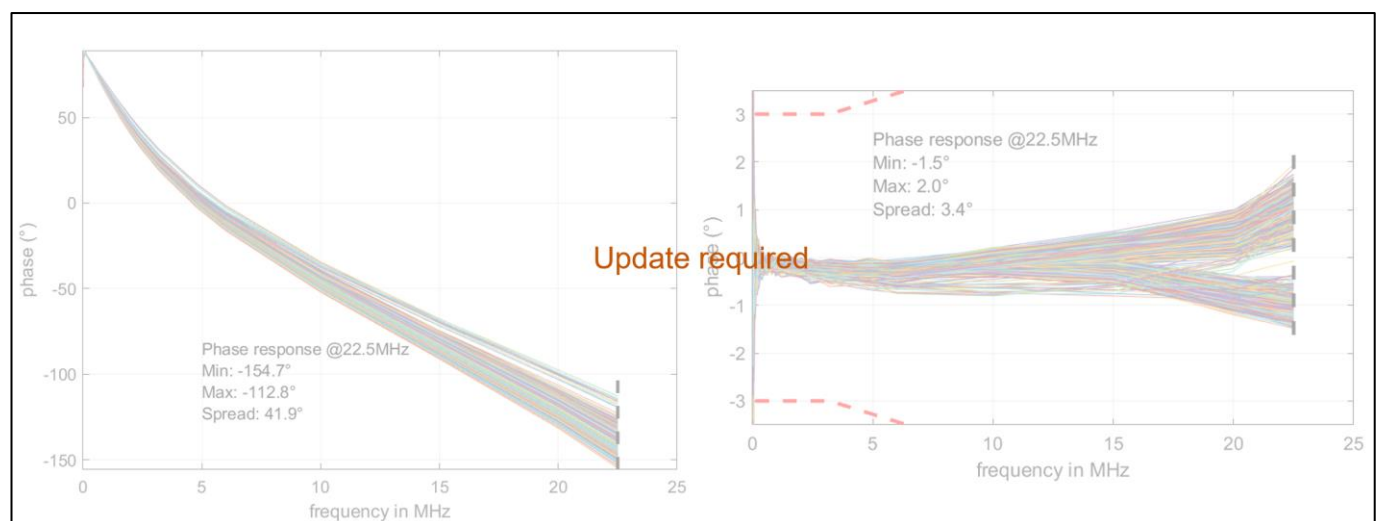


Figure 7: RX phase response without (left) and with (right) compensation of process variation

1.4.1 Amplitude response

The device has a 2nd order analog high pass (AHP) filter with configurable cut-off frequencies.

CTRX8188F - Cascading

The formula (1) describes the amplitude response of a dedicated MMIC with the manufacturing process variation included, if f_{c_AHP} represents the corner frequency calculated from the data stored inside each MMIC and returned by the firmware command *Get_RX_TF_Parameters()* (see User Manual [1]).

$$A_{model_BBF}(f_{IF}, f_{c_AHP}) = 40 \cdot \log_{10} \left(\frac{f_{IF}/f_{c_AHP}}{\sqrt{1 + (f_{IF}/f_{c_AHP})^2}} \right) \quad (1)$$

The process variation of the amplitude response of the Anti-Aliasing Filter (AAF, Low Pass Filter) can be neglected.

1.4.2 Phase response

The phase response of the analog baseband filter with the manufacturing process variation of a dedicated MMIC is given by $\varphi_{model_BBF}(f_{IF}, t_{BBF}, f_{c_AHP})$. It is defined as the sum of the scaled Anti-aliasing Filter (AAF) and the scaled analog high pass filter phase response models:

$$\varphi_{model_BBF}(f_{IF}, t_{BBF}, f_{c_AHP}) = \varphi_{model_AAF_scaled}(f_{IF}, t_{BBF}) + \varphi_{model_AHP}(f_{IF}, f_{c_AHP}) \quad (2)$$

The scaled AAF phase response model $\varphi_{model_AAF_scaled}(f_{IF}, t_{BBF})$ is given by:

$$\varphi_{model_AAF_scaled}(f_{IF}, t_{BBF}) = \varphi_{model_AAF} \left(f_{IF} \times \frac{t_{BBF}}{t_{model_BBF}} \right) \quad (3)$$

where $t_{model_BBF} = t_{bd}$ ns, and t_{BBF} is the baseband filter delay, measured as average group delay between 100 kHz and 15 MHz of the baseband filter, excluding the impact of AHP phase response. It is stored into each MMIC and returned by the firmware command *Get_RX_TF_Parameters()* (see User Manual [1]).

The AAF phase response model $\varphi_{model_AAF}(f_{model_AAF})$ is given in the table below with linear interpolation between frequency points.

Table 2: Anti-aliasing filter phase response model

f_{model_AAF} [MHz]	φ_{model_AAF} [°]
0.0	tbd
15.0	tbd
18.3	tbd
20.0	tbd
22.5	tbd
24.0	tbd
26.0	tbd
28.0	tbd

The scaled phase response model $\varphi_{model_AHP}(f_{IF}, f_{c_AHP})$ of the AHP filter is given by:

$$\varphi_{model_AHP}(f_{IF}, f_{c_AHP}) = 180^\circ - 2 \cdot \text{atan} \left(\frac{f_{IF}}{f_{c_AHP}} \right) \quad (4)$$

CTRX8188F - Cascading

where f_{c_AHP} is the measured 6 dB attenuation corner frequency of the AHP.

1.4.3 Implementation

The firmware command *Get_RX_TF_Parameters()* returns the deviation of the corner frequency for the AHP and the AAF baseband delay for two temperatures T1 and T2 (close to +135°C and -40°C, exact values returned by firmware command *Get_RX_TF_Parameters()*) in percentage. The deviation can be calculated for every temperature by linear interpolation.

These are the steps to be implemented in the MCU/CPU to compensate the RX analog frontend process variations of the MMICs in a cascaded system:

1. Get deviation d_{fc} for the corner frequency f_{c_AHP} and the deviation d_{tBBF} for baseband delay for t_{BBF} for temperature T1 and T2 in percentage with the firmware command *Get_RX_TF_Parameters()*
2. Interpolate the deviation d_{fc} and d_{tBBF} for the required temperature T to get d_{fc_T} and d_{tBBF_T}
3. Calculate corner frequency f_{c_AHP} and baseband delay t_{BBF}

$$a. \quad f_{c_AHP} = f_{c_AHP \text{ nominal}} \times \left(1 + \frac{d_{fc_T}}{100}\right)$$

$$b. \quad t_{BBF} = t_{\text{model_BBF}} \times \left(1 + \frac{d_{tBBF_T}}{100}\right)$$

4. Calculate
 - a. real amplitude response $A_{\text{model_BBF}}$ with (1) or
 - b. real phase response $\varphi_{\text{model_BBF}}$ with (2), (3), (4) and
 - c. [Table 2](#)

5. Use the results from point 4. to compensate process variations in post processing in MCU/CPU

CTRX8188F - Cascading

2 Firmware configuration flow

This chapter describes the firmware configuration used for PRIMARY and SECONDARY MMICs. Here, the start-up sequence as well as a full application cycle including calibration, sequencing and monitoring is considered.

Generally, the same start-up sequence is used for PRIMARY and SECONDARY MMICs. The start-up sequence itself is described in the User Manual [1]. At this point, it is assumed that each MMIC is already in *Pre_Operation* state running on its internally generated clock.

Note: *There is no hard timing to be considered, unless explicitly mentioned. Nevertheless, the order of consecutive executable firmware commands has to be ensured. In this application note, the order is always highlighted.*

In [Figure 8](#), an overview of the firmware configuration flow needed to set up a cascaded system is shown. Based on the application example the corresponding flow is discussed step-by-step.

Note: *The basic firmware functionality (e.g. `Configure_RX()` ...), which is not cascading-specific, is described in the User Manual [1], and therefore not part of the description.*

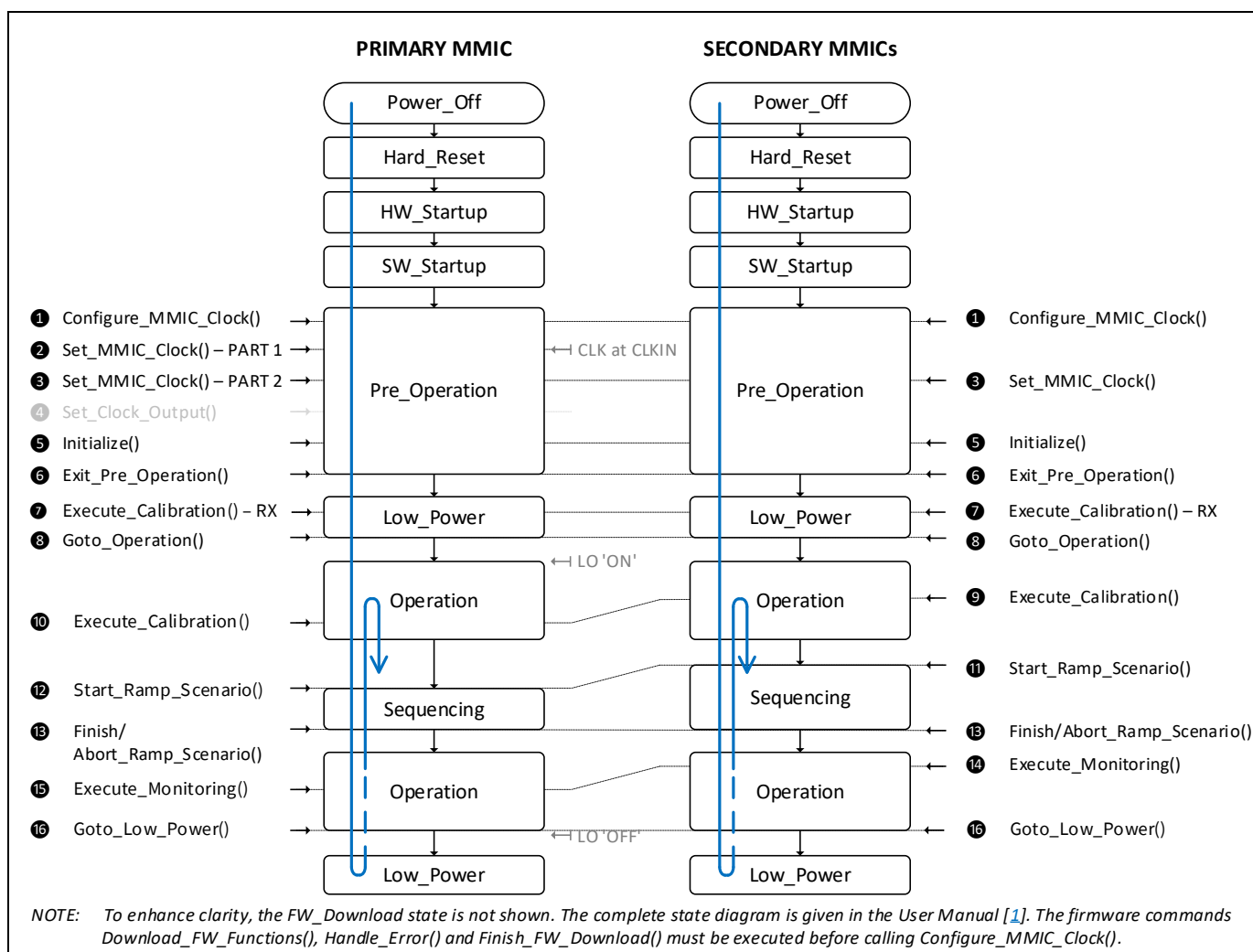


Figure 8: Firmware configuration flow (application example)

CTRX8188F - Cascading

2.1 50 MHz clock distribution

The clock self-feeding (see ❶ to ❸) is used to ensure common 50 MHz clock, provided by PRIMARY MMIC to each MMIC and therefore time-wise accurate sampling. The configuration described below has to be done by the system integrator in *Pre_Operation* state.

Step 1: Clock Distribution Configuration

The clock self-feeding has to be configured by the system integrator. Therefore, there is the firmware command *Configure_MMIC_Clock()*. The firmware command is executable at PRIMARY and SECONDARY MMICs in any order. The parameter value to be used is stated below.

PRIMARY MMIC (MMIC 1)

For the PRIMARY MMIC, the *clock configuration selection* is set to *PRIMARY* clock configuration: 7_D.

```
Configure_MMIC_Clock(< ... 7D >)
```

SECONDARY MMIC (MMIC 2 - 4)

For the SECONDARY MMICs, the *clock configuration selection* is set to *SECONDARY* clock configuration: 2_D.

```
Configure_MMIC_Clock(< ... 2D >)
```

Note: According to the User Manual [1] CLKOUT load bias voltage and CLKOUT source resistance are optionally configurable. For PRIMARY MMIC CLKOUT source resistance is set to 40 Ω (see Figure 5).

Step 2: Clock Distribution Enablement

To enable clock distribution, CLKOUT and CLKIN are enabled by executing *Set_MMIC_Clock()*.

PRIMARY MMIC (MMIC 1)

First, the system integrator has to ensure that the clock at CLKIN is stable by executing the firmware command *Set_MMIC_Clock()* with the *action mask* 1_D (= first part).

```
Set_MMIC_Clock(< ... 1D >)
```

After that, the firmware command *Set_MMIC_Clock()* can be executed with the action mask 2_D (second part).

```
Set_MMIC_Clock(< ... 2D >)
```

SECONDARY MMIC (MMIC 2 - 4)

For the SECONDARY MMICs, the firmware command *Set_MMIC_Clock()* has to be executed with the action mask 3_D. The firmware command has to be executed either after completing the second execution of the firmware command *Set_MMIC_Clock()* for the PRIMARY MMIC or in parallel.

```
Set_MMIC_Clock(< ... 3D >)
```

2.2 Providing clock source to peripheral (optional, only for PRIMARY MMIC)

The device is designed to serve as single clock source for other peripherals like a microcontroller (see ❹).

Step 1: Configure CLKMCU

The CLKMCU output pin configuration can be configured with firmware command *Configure_DIOs()*. A detailed description is given in the User Manual [1].

CTRX8188F - Cascading

Step 2: Enable CLKMCU and/or CLKPHY

At the PRIMARY MMIC the *CLKMCU* output and the *CLKPHY* output can be activated and deactivated by executing the firmware command *Set_Clock_Output()* with the corresponding *action field* set to 1_B or 0_B. In addition, the frequency of the *CLKMCU* output can be selected. A description is given in the User Manual [1].

Note: After power-up sequence or a hard-reset event the *CLKMCU* output and the *CLKPHY* output are deactivated.

2.3 LO distribution

The LO distribution is used to ensure a common LO signal provided by PRIMARY MMIC to each MMIC.

Step 1: LO Distribution Configuration

The LO configuration stored in the *default settings table* is configurable via executing the firmware command *Initialize()* in *Pre_Operation* state (see 5). The *default settings table* is described in detail in User Manual [1].

Note: By default, after powering-up or performing a hard reset, the MMIC is configured as *STAND-ALONE MMIC*.

PRIMARY MMIC (MMIC 1)

At the PRIMARY MMIC the LO signal generated by digital phase locked loop (DPLL) has to be provided at *LOOUT* port. Therefore, DPLL and *LOOUT* port has to be enabled. For LO self-feeding in addition *LOIN* port has to be selected, and finally enabled. In this example, the system integrator has to select *PRIMARY using LOIN2*: 15_D.

<LO configuration> = 15_D

In addition, system integrator has to set an LO input power target value. As described in User Manual [1], the LO output power is adjusted correspondingly. In this example, *LOIN* power target value is assumed to be 1 dBm (default).

<LOIN power target value> = 128_D

SECONDARY MMIC (MMIC 3)

At the SECONDARY MMIC the *LOIN* port has to be enabled. In this example, the system integrator has to select *SECONDARY using LOIN2*: 12_D.

<LO configuration> = 12_D

SECONDARY MMIC (MMIC 2, MMIC 4)

At the SECONDARY MMICs the *LOIN* port has to be enabled. In this example, the system integrator has to select *SECONDARY using LOIN1*: 4_D.

<LO configuration> = 4_D

By executing the firmware command *Exit_Pre_Operation()* the *Pre_Operation* state is left (see 6). The execution can be done in any order at PRIMARY and SECONDARY MMICs.

CTRX8188F - Cascading

Step 2: LO Distribution Enablement

By executing the firmware command *Goto_Operation()*, the LO is activated according to the selected configuration (see 8). The LO signal is provided at LOOUT as soon as entering *Operation* state at PRIMARY MMIC.

Note: By executing firmware command *Goto_Low_Power()*, LO configuration is de-activated.

2.4 Calibration

The MMIC calibration is performed in the *Operation* state whereas the receiver calibration can also be done in *Low_Power* state (see 9 to 10). The synchronization between the MMICs (valid for transmitter calibration) is handled via a trigger modulated on top of the common clock signal via DMUX3 as described in [Chapter 1.3](#).

First, the firmware command *Execute_Calibration()* has to be called at SECONDARY MMICs, then at PRIMARY MMIC. In the end, the PRIMARY MMIC initiate the trigger event. It is required that each SECONDARY MMIC is ready to receive the clock trigger event. As described in [Chapter 1.3](#) this is ensured via DMUX3. After the synchronous calibration has been finished the command response of each MMIC can be read. The firmware command *Execute_Calibration()* is described in the User Manual [1].

Generally, the system integrator can select each calibration sub-function to be executed individually. In [Table 3](#) an overview of each sub-function and its time dependency with respect to execution on PRIMARY and SECONDARY MMIC is given. A detailed overview is given in the User Manual [1].

Note: Except for LO input power calibration, the same calibration sub-functions and same number of channels need to be selected.

Note: It is recommended to execute RX related calibration sub-functions in *Low_Power* state 7: RX BBADC calibration and RX gain mapping and temperature compensation. RX BBADC calibration and RX gain mapping and temperature compensation are executable simultaneously for PRIMARY MMICs and SECONDARY MMICs.

Table 3: Usage of calibration sub-functions in a cascaded setup

Calibration sub-functions	PRIMARY MMIC	SECONDARY MMIC	Note
LO input power calibration	yes	n/a	-
TX phase power-up calibration, part 1	yes	yes	Call <i>Execute_Calibration()</i> first at SECONDARY MMICs, then at PRIMARY MMIC; the calibration is initiated by the PRIMARY MMIC (see Chapter 1.3)
TX phase power-up calibration, part 2	yes	yes	
TX power calibration	yes	yes	
TX phase warm-up calibration	yes	yes	
RX BBADC calibration A, step 1	yes	yes	Call <i>Execute_Calibration()</i> in any order wrt. PRIMARY and SECONDARY MMICs. Execution is recommended in the <i>Low_Power</i> state; <i>Operation</i> state allowed.
RX BBADC calibration A, step 2	yes	yes	
RX BBADC calibration B	yes	yes	
RX gain mapping and temperature compensation	yes	yes	

CTRX8188F - Cascading

2.5 Sequencing

The firmware command `Start_Ramp_Scenario()` is used to initiate the execution of a pre-defined frequency ramp scenario. By executing the firmware command `Finish_Ramp_Scenario()` the transition to the Operation state is triggered (see 11 to 13). A detailed description is given in the User Manual [1]. The synchronization between PRIMARY MMIC and SECONDARY MMICs is handled via a trigger modulated on top of the common clock as described in Chapter 1.3.

First, firmware command `Start_Ramp_Scenario()` has to be executed at SECONDARYs, then at PRIMARY MMIC. In the end, the PRIMARY MMIC initiates the trigger event.

Note: To initiate the execution of a ramp scenario via a hardware trigger, the sequencer program has to start with a wait opcode. Among others, the `[SRC_DMUX]` bit fields have to be set accordingly. Further information is given in the User Manual [1].

Note: The first payload ramp segment after a CLKIN trigger event must not start before the system clock has settled, please refer to the parameter $t_{sys_CLK_set}$ specified in the Data Sheet [2] in order to settle the internal clock. For that, it is recommended to use a "wait segment" as first ramp segment after a clock trigger as illustrated in Figure 9. The following settings need to be set in the `OP_WAIT` opcode of the sequencer program: `REDIR_GRP1_CLKOUT` for PRIMARY MMIC, `SRC_CASC_CLKIN` for PRIMARY and SECONDARY MMICs. A description is given in the User Manual [1].

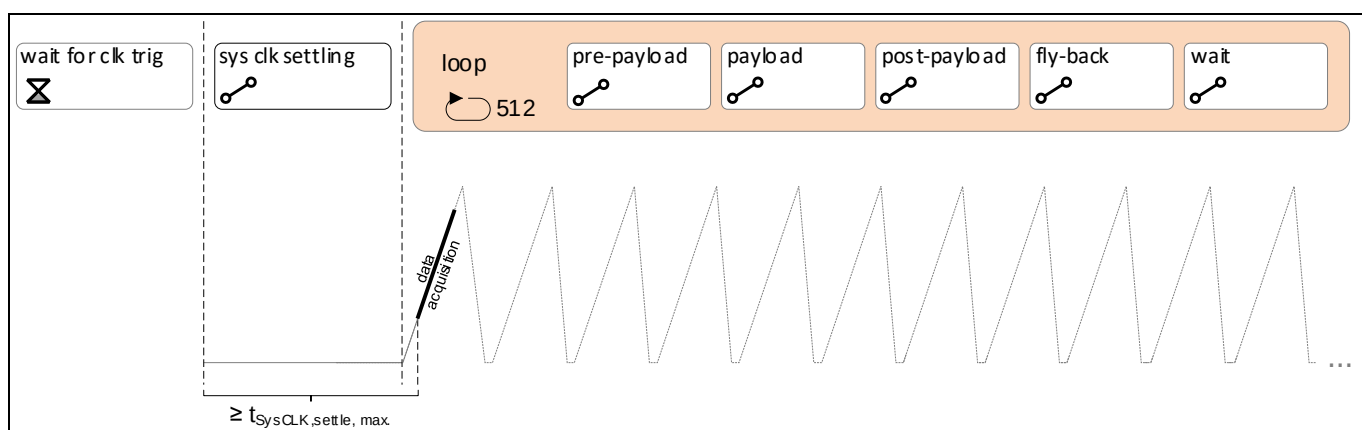


Figure 9: Example ramp sequence (application example)

2.6 Monitoring

The firmware command `Execute_Monitoring()` is used to perform MMIC monitoring (see 14 to 15). A detailed description is given in the User Manual [1]. The synchronization between PRIMARY MMIC and SECONDARY MMICs is handled via a trigger modulated on top of the common clock signal and via DMUX3 as described in Chapter 1.3.

First, firmware command `Execute_Monitoring()` has to be called at SECONDARYs, then at PRIMARY MMIC. In the end, the PRIMARY MMIC initiates the trigger event. After the synchronous monitoring has been finished the command response of the PRIMARY MMIC and each SECONDARY MMIC can be read.

Generally, the system integrator can select each monitoring sub-function to be executed individually. In Table 4 an overview of each sub-function and its time dependency with respect to execution on PRIMARY and SECONDARY MMIC is given. A detailed overview is given in the User Manual [1].

Note: The same monitoring sub-functions and same number of channels need to be selected for all MMICs.

CTRX8188F - Cascading

Table of contents

Table 4: Usage of monitoring sub-functions in a cascaded setup

Monitoring sub-routine	PRIMARY MMIC	SECONDARY MMICs	Note
TX phase balance monitoring	yes	yes	Call <code>Execute_Monitoring()</code> first at SECONDARY MMICs, then at PRIMARY MMIC; the monitoring is initiated by PRIMARY MMIC (Chapter 1.3)
TX signal power monitoring	yes	yes	
TX power balance monitoring	yes	yes	
RX RF monitoring	yes	yes	
LO solder connection monitoring	yes	yes	Call <code>Execute_Monitoring()</code> in any order wrt. PRIMARY and SECONDARY MMICs.
RX baseband monitoring	yes	yes	
Register content integrity check	yes	yes	

After monitoring is completed, the `Low_Power` state can be entered by executing `Goto_Low_Power()` (see [16](#)).

2.7 Pseudocode

Below is an example how to configure PRIMARY MMIC and SECONDARY MMIC as part of a cascaded system as depicted in [Figure 1](#). This example can be optimized and adjusted depending on the customer's use case and assumptions as temperature profile and so on. For the example pseudocode is used. More detailed information on each individual firmware command including a full description of input/output parameters is given in the User Manual [\[1\]](#). Here, only cascaded-specific parameters are highlighted.

As described in the User Manual [\[1\]](#), there is the option to use e.g. a batch command. The option is highlighted in the function field, e.g. `[BATCH 1]`.

Note: Unless otherwise stated, each firmware command can be executed in any order according to PRIMARY and SECONDARY MMICs.

Code Listing

```
...

// read previous error status (optional)
MMIC1.Handle_Reset_Reason(...)
MMIC2.Handle_Reset_Reason(...)
MMIC3.Handle_Reset_Reason(...)
MMIC4.Handle_Reset_Reason(...)
// trigger logical built-in self-test (LBIST)
MMIC1.Trigger_LBIST(...)
MMIC2.Trigger_LBIST(...)
MMIC3.Trigger_LBIST(...)
MMIC4.Trigger_LBIST(...)

// read previous error status (optional)
MMIC1.Handle_Reset_Reason(...)
MMIC2.Handle_Reset_Reason(...)
MMIC3.Handle_Reset_Reason(...)
MMIC4.Handle_Reset_Reason(...)
```


CTRX8188F - Cascading

Table of contents

Code Listing

```
// download required RAM FW
MMIC1.Download_FW_Functions(...) // n times, depending on RAM firmware size
MMIC2.Download_FW_Functions(...) // n times, depending on RAM firmware size
MMIC3.Download_FW_Functions(...) // n times, depending on RAM firmware size
MMIC4.Download_FW_Functions(...) // n times, depending on RAM firmware size

// error check
// if an error status is logged that is not whitelisted, the system
// integrator shall trigger a hard reset, see User Manual [1]; special
// fault handling
MMIC1.Handle_Error(...) // read all
MMIC2.Handle_Error(...) // read all
MMIC3.Handle_Error(...) // read all
MMIC4.Handle_Error(...) // read all

// finish RAM FW download
MMIC1.Finish_FW_Download(...)
MMIC2.Finish_FW_Download(...)
MMIC3.Finish_FW_Download(...)
MMIC4.Finish_FW_Download(...)

// ❶ configure MMIC clock
// PRIMARY MMIC/SECONDARY MMICs
MMIC1.clock_configuration_selection = 7b
MMIC2.clock_configuration_selection = 2b
MMIC3.clock_configuration_selection = 2b
MMIC4.clock_configuration_selection = 2b

MMIC1.Configure_MMIC_Clock(MMIC1.clock_configuration_selection, ...)
MMIC2.Configure_MMIC_Clock(MMIC2.clock_configuration_selection, ...)
MMIC3.Configure_MMIC_Clock(MMIC3.clock_configuration_selection, ...)
MMIC4.Configure_MMIC_Clock(MMIC4.clock_configuration_selection, ...)

// ❷ to ❸ set MMIC clock (50-MHz)
// PRIMARY MMIC
MMIC1.action_mask = 1b

MMIC1.Set_MMIC_Clock(MMIC1.action_mask)
// PRIMARY MMIC/SECONDARY MMICs
MMIC1.action_mask = 2b
MMIC2.action_mask = 3b
MMIC3.action_mask = 3b
MMIC4.action_mask = 3b

MMIC1.Set_MMIC_Clock(MMIC1.action_mask)
MMIC2.Set_MMIC_Clock(MMIC2.action_mask)
MMIC3.Set_MMIC_Clock(MMIC3.action_mask)
MMIC4.Set_MMIC_Clock(MMIC4.action_mask)

// ❹ configure and set clock output (optional)
// configure DMUXs (optional)
MMIC1.Configure_DMUX(...)
MMIC2.Configure_DMUX(...)
```

CTRX8188F - Cascading

Table of contents

Code Listing

```

MMIC3.Configure_DMUX(...)
MMIC4.Configure_DMUX(...)

// configure digital I/Os
MMIC1.Configure_DIOs(...)
MMIC2.Configure_DIOs(...)
MMIC3.Configure_DIOs(...)
MMIC4.Configure_DIOs(...)

// PRIMARY MMIC
MMIC1.Set_Clock_Output(...)

// ⑤      configure LO, ...
MMIC1.LO_config = 15D
MMIC2.LO_config = 12D // LOIN 2
MMIC3.LO_config = 4D // LOIN 1
MMIC4.LO_config = 12D // LOIN 2

MMIC1.LOIN_pwr_target = 128D

MMIC1.Initialize(..., 10000028H, MMIC1.LO_config, ...
                  A1000028H, MMIC1.LOIN_pwr_target, ...)
MMIC2.Initialize(..., 10000028H, MMIC2.LO_config, ...)
MMIC3.Initialize(..., 10000028H, MMIC3.LO_config, ...)
MMIC4.Initialize(..., 10000028H, MMIC4.LO_config, ...)

// ⑥      trigger transition into Low_Power state
MMIC1.Exit_Pre_Operation(...)
MMIC2.Exit_Pre_Operation(...)
MMIC3.Exit_Pre_Operation(...)
MMIC4.Exit_Pre_Operation(...)

// clear previous error status (optional)
MMIC1.Handle_Reset_Reason(...)
MMIC2.Handle_Reset_Reason(...)
MMIC3.Handle_Reset_Reason(...)
MMIC4.Handle_Reset_Reason(...)

// define frequency band and static frequency for power-up calibration
// frequency range must include complete frequency range, which is used in
// all application cycles
// lock frequency (NCW) and frequency bandwidth (RAMPBW) must configured to
// same values for all MMICs
MMIC1.Configure_RF_Frequency(...)
MMIC2.Configure_RF_Frequency(...)
MMIC3.Configure_RF_Frequency(...)
MMIC4.Configure_RF_Frequency(...)

// configure RX
MMIC1.Configure_RX(...)
MMIC2.Configure_RX(...)
MMIC3.Configure_RX(...)
MMIC4.Configure_RX(...)

```

CTRX8188F - Cascading

Table of contents

Code Listing

```
// configure slope scaling factor for calibration
MMIC1.Configure_TX_Power()
MMIC2.Configure_TX_Power()
MMIC3.Configure_TX_Power()
MMIC4.Configure_TX_Power()

// trigger transition into Operation state
MMIC1.Goto_Operation(...)
MMIC2.Goto_Operation(...)
MMIC3.Goto_Operation(...)
MMIC4.Goto_Operation(...)

// perform power-up calibration, clock-triggered
// please refer to the User Manual [1]
MMIC1.calib_sub_func_ID = 0000000000011100B // PRIMARY MMIC: Lo input power
MMIC2.calib_sub_func_ID = 0000000000001100B
MMIC3.calib_sub_func_ID = 0000000000001100B
MMIC4.calib_sub_func_ID = 0000000000001100B

// SECONDARY MMICs
MMIC2.Execute_Calibration(MMIC2.calib_sub_func_ID, ...)
MMIC3.Execute_Calibration(MMIC3.calib_sub_func_ID, ...)
MMIC4.Execute_Calibration(MMIC4.calib_sub_func_ID, ...)

// PRIMARY MMIC
MMIC1.Execute_Calibration(MMIC1.calib_sub_func_ID, ...)

// trigger transition into Low_Power state
MMIC1.Goto_Low_Power(...)
MMIC2.Goto_Low_Power(...)
MMIC3.Goto_Low_Power(...)
MMIC4.Goto_Low_Power(...)

// start radar application cycle

// download sequencer program(s) | PRIMARY MMIC and SECONDARY MMICs
MMIC1.rmp_file = ...
MMIC2.rmp_file = ...
MMIC3.rmp_file = ...
MMIC4.rmp_file = ...
MMIC1.Write_Memory(..., MMIC1.rmp_file)
MMIC2.Write_Memory(..., MMIC2.rmp_file)
MMIC3.Write_Memory(..., MMIC3.rmp_file)
MMIC4.Write_Memory(..., MMIC4.rmp_file)

// Run BATCH 1 by executing Run_Batch()

// [BATCH 1] clear result area for monitoring
MMIC1.Clear_Results(...)
MMIC2.Clear_Results(...)
MMIC3.Clear_Results(...)
MMIC4.Clear_Results(...)
```

CTRX8188F - Cascading

Table of contents

Code Listing

```
// [BATCH 1] configure ramp scenario / select sequencer program
MMIC1.Configure_Ramp_Scenario(...)
MMIC2.Configure_Ramp_Scenario(...)
MMIC3.Configure_Ramp_Scenario(...)
MMIC4.Configure_Ramp_Scenario(...)

// [BATCH 1] configure RF frequency / define frequency band and static
// frequency for the application cycle for the selected ramp scenario
MMIC1.Configure_RF_Frequency(...)
MMIC2.Configure_RF_Frequency(...)
MMIC3.Configure_RF_Frequency(...)
MMIC4.Configure_RF_Frequency(...)

// [BATCH 1] configure TX power
// note: invalidates last TX power and phase warm-up calibration
MMIC1.Configure_TX_Power(...)
MMIC2.Configure_TX_Power(...)
MMIC3.Configure_TX_Power(...)
MMIC4.Configure_TX_Power(...)

// [BATCH 1] configure RX
MMIC1.Configure_RX(...)
MMIC2.Configure_RX(...)
MMIC3.Configure_RX(...)
MMIC4.Configure_RX(...)

// [BATCH 1] ⑦ execute warm-up calibration (RX sub-calibration)
MMIC1.calib_sub_func_ID = 0011011000000000B
MMIC2.calib_sub_func_ID = 0011011000000000B
MMIC3.calib_sub_func_ID = 0011011000000000B
MMIC4.calib_sub_func_ID = 0011011000000000B

// PRIMARY/ SECONDARY MMICs
MMIC1.Execute_Calibration(MMIC1.calib_sub_func_ID, ...)
MMIC2.Execute_Calibration(MMIC2.calib_sub_func_ID, ...)
MMIC3.Execute_Calibration(MMIC3.calib_sub_func_ID, ...)
MMIC4.Execute_Calibration(MMIC4.calib_sub_func_ID, ...)

// [BATCH 1] read RX calibration temperatures
MMIC1.Get_Temperature(...)
MMIC2.Get_Temperature(...)
MMIC3.Get_Temperature(...)
MMIC4.Get_Temperature(...)

// [BATCH 1] ⑧ trigger transition into Operation state and read DPLL
// calibration temperature
// [BATCH 1] PRIMARY MMIC
MMIC1.Goto_Operation(...)
MMIC2.Goto_Operation(...)
MMIC3.Goto_Operation(...)
MMIC4.Goto_Operation(...)

MMIC1.Get_Temperature(...)
```

CTRX8188F - Cascading

Table of contents

Code Listing

```
// [BATCH 1] ⑨ to ⑩      execute warm-up calibration, clock-triggered (TX
// sub-calibration)
MMIC1.calib_sub_func_ID = 0000000000000011B
MMIC2.calib_sub_func_ID = 0000000000000011B
MMIC3.calib_sub_func_ID = 0000000000000011B
MMIC4.calib_sub_func_ID = 0000000000000011B

// SECONDARY MMICs
MMIC2.Execute_Calibration(MMIC2.calib_sub_func_ID, ...)
MMIC3.Execute_Calibration(MMIC3.calib_sub_func_ID, ...)
MMIC4.Execute_Calibration(MMIC4.calib_sub_func_ID, ...)

// PRIMARY MMIC
MMIC1.Execute_Calibration(MMIC1.calib_sub_func_ID, ...)

// [BATCH 1] read TX calibration temperatures
MMIC1.Get_Temperature(...)
MMIC2.Get_Temperature(...)
MMIC3.Get_Temperature(...)
MMIC4.Get_Temperature(...)

// [BATCH 1] configure slow slope scaling factor for TX PA power up
// (optional)
MMIC1.Configure_TX_Power(...)
MMIC2.Configure_TX_Power(...)
MMIC3.Configure_TX_Power(...)
MMIC4.Configure_TX_Power(...)

// [BATCH 1] enable TX power amplifier with internal dump (optional)
MMIC1.Set_TX_Output(...)
MMIC2.Set_TX_Output(...)
MMIC3.Set_TX_Output(...)
MMIC4.Set_TX_Output(...)

// [BATCH 1] configure slope scaling factor for ramp scenario (optional)
MMIC1.Configure_TX_Power(...)
MMIC2.Configure_TX_Power(...)
MMIC3.Configure_TX_Power(...)
MMIC4.Configure_TX_Power(...)

// [BATCH 1] synchronize MMICs via Wait()
// NOTE: wait time for PRIMARY MMIC after synchronization has to be
// considered, please refer to the User Manual [1]
MMIC1.Wait(...)
MMIC2.Wait(...)
MMIC3.Wait(...)
MMIC4.Wait(...)

// [BATCH 1] ⑪ to ⑫      execute ramp scenario
// SECONDARY MMICs
MMIC2.Start_Ramp_Scenario(...)
MMIC3.Start_Ramp_Scenario(...)
MMIC4.Start_Ramp_Scenario(...)
```

CTRX8188F - Cascading

Table of contents

Code Listing

```
// PRIMARY MMIC
MMIC1.Start_Ramp_Scenario(...)

// [BATCH 1] read temperature (pre-ramp scenario)
MMIC1.Get_Temperature(...)
MMIC2.Get_Temperature(...)
MMIC3.Get_Temperature(...)
MMIC4.Get_Temperature(...)

// [BATCH 1] 13 finish ramp scenario (call after ramp scenario finished)
MMIC1.Finish_Ramp_Scenario(...)
MMIC2.Finish_Ramp_Scenario(...)
MMIC3.Finish_Ramp_Scenario(...)
MMIC4.Finish_Ramp_Scenario(...)

// [BATCH 1] read temperature (post ramp scenario)
MMIC1.Get_Temperature(...)
MMIC2.Get_Temperature(...)
MMIC3.Get_Temperature(...)
MMIC4.Get_Temperature(...)

// [BATCH 1] configure slow slope scaling factor for TX PA power down
// (optional)
MMIC1.Configure_TX_Power(...)
MMIC2.Configure_TX_Power(...)
MMIC3.Configure_TX_Power(...)
MMIC4.Configure_TX_Power(...)

// [BATCH 1] disable TX power amplifier (optional)
MMIC1.Set_TX_Output(...)
MMIC2.Set_TX_Output(...)
MMIC3.Set_TX_Output(...)
MMIC4.Set_TX_Output(...)

// [BATCH 1] configure slope scaling factor for monitoring/calibration
// (optional)
MMIC1.Configure_TX_Power(...)
MMIC2.Configure_TX_Power(...)
MMIC3.Configure_TX_Power(...)
MMIC4.Configure_TX_Power(...)

// compare temperature readings from pre and post ramp scenario against
// temperature readings from DPLL, TX and RX calibration so the calibration
// requirements from safety manual are satisfied.

// [BATCH 1] 14 to 15 execute monitoring, clock-triggered
// (ISM_TX_REL_PHASE: NPHASE, monitoring phase 1, see User Manual [1] and
// Safety Manual [])
MMIC1.mon_sub_func_ID = 0000000000000001B
MMIC2.mon_sub_func_ID = 0000000000000001B
MMIC3.mon_sub_func_ID = 0000000000000001B
MMIC4.mon_sub_func_ID = 0000000000000001B
```

CTRX8188F - Cascading

Table of contents

Code Listing

```
// SECONDARY MMICs
MMIC2.Execute_Monitoring(MMIC2.mon_sub_func_ID, ...)
MMIC3.Execute_Monitoring(MMIC3.mon_sub_func_ID, ...)
MMIC4.Execute_Monitoring(MMIC4.mon_sub_func_ID, ...)

// PRIMARY MMIC
MMIC1.Execute_Monitoring(MMIC1.mon_sub_func_ID, ...)

// [BATCH 1] ⑭ to ⑮      execute monitoring, clock-triggered
// (ISM_TX_REL_PHASE: NPHASE, monitoring phase 2, see User Manual [1] and
// Safety Manual [])
// sub-function ID depends on customer use case, see Safety Manual []
MMIC1.mon_sub_func_ID = 000001101010111B
MMIC2.mon_sub_func_ID = 000001101010111B
MMIC3.mon_sub_func_ID = 000001101010111B
MMIC4.mon_sub_func_ID = 000001101010111B

// SECONDARY MMICs
MMIC2.Execute_Monitoring(MMIC2.mon_sub_func_ID, ...)
MMIC3.Execute_Monitoring(MMIC3.mon_sub_func_ID, ...)
MMIC4.Execute_Monitoring(MMIC4.mon_sub_func_ID, ...)

// PRIMARY MMIC
MMIC1.Execute_Monitoring(MMIC1.mon_sub_func_ID, ...)

// [BATCH 1] get monitoring info | PRIMARY MMIC and SECONDARY MMICs
// (optional)
MMIC1.Get_Results(...)
MMIC2.Get_Results(...)
MMIC3.Get_Results(...)
MMIC4.Get_Results(...)

// [BATCH 1]  to ⑯      transition to Low_Power state
MMIC1.Goto_Low_Power(...)
MMIC2.Goto_Low_Power(...)
MMIC3.Goto_Low_Power(...)
MMIC4.Goto_Low_Power(...)

// [BATCH 1] read error information - alternatively the error information
// can be streamed out using CSI-2 via Get_Result() in operation state
// after step ⑯
// if an error status is logged, perform fault handling as described in
// User Manual [1] and Safety Manual []
MMIC1.Handle_Error(...)
MMIC2.Handle_Error(...)
MMIC3.Handle_Error(...)
MMIC4.Handle_Error(...)

// read BATCH 1 results | PRIMARY MMIC and SECONDARY MMICs
MMIC1.Get_Results(...)
MMIC2.Get_Results(...)
MMIC3.Get_Results(...)
MMIC4.Get_Results(...)
```

CTRX8188F - Cascading

Table of contents

Code Listing

```
// continue with next radar cycle ...  
// continue with "download sequencer program(s)" as described above
```


CTR8188F - Cascading

Table of contents

3 Terminology

A	
ADC	Analog to Digital Converter
B	
C	
CLK	Clock
D	
DMUX	Digital Multiplexer
DPLL	Digital Phase Locked Loop
E	
F	
FW	Firmware
G	
GND	Ground
H	
I	
J	
K	
L	
LBIST	Logic Built-in Self-Test
LO	Local Oscillator
M	
MCU	Micro Controller Unit
MMIC	Monolithic Microwave Integrated Circuit
N	
O	
P	
PA	Power Amplifier
PCB	Printed Circuit Board
Q	
R	
RX	Receiver/Receive



CTR8188F - Cascading

Table of contents

S	
T	
TF2BGA	Thin-Profile Fine-Pitch 2 Ball Grid Array
TX	Transmitter/Transmit
U	
V	
W	
X	
Y	
Z	

CTRX8188F - Cascading

Table of contents

4 References

The following documents provide more information. For Infineon documents related to the device, please refer to the applicable revision, as communicated by Infineon separately.

[1] User Manual CTRX8188F

[2] CTRX8188F Target Datasheet

[3] CTRX8188F Safety Manual

[4] CTRX8188F/FS Safety Package Release Note

CTR8188F - Cascading

Table of contents

5 Revision history

Table 5: Revision history

Document version	Date of release	Description of changes
0.10	17-06-2025	Initial release

Trademarks

All referenced product or service names and trademarks are the property of their respective owners.

Edition 2025-06-17

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2025 Infineon Technologies AG.

All Rights Reserved.

Do you have a question about this document?

Email: erratum@infineon.com

Document reference

Z8F80812891

IMPORTANT NOTICE

The information contained in this application note is given as a hint for the implementation of the product only and shall in no event be regarded as a description or warranty of a certain functionality, condition or quality of the product. Before implementation of the product, the recipient of this application note must verify any function and other technical information given herein in the real application. Infineon Technologies hereby disclaims any and all warranties and liabilities of any kind (including without limitation warranties of non-infringement of intellectual property rights of any third party) with respect to any and all information given in this application note.

The data contained in this document is exclusively intended for technically trained staff. It is the responsibility of customer's technical departments to evaluate the suitability of the product for the intended application and the completeness of the product information given in this document with respect to such application.

For further information on the product, technology, delivery terms and conditions and prices please contact your nearest Infineon Technologies office (www.infineon.com).

WARNINGS

Due to technical requirements products may contain dangerous substances. For information on the types in question please contact your nearest Infineon Technologies office.

Except as otherwise explicitly approved by Infineon Technologies in a written document signed by authorized representatives of Infineon Technologies, Infineon Technologies' products may not be used in any applications where a failure of the product or any consequences of the use thereof can reasonably be expected to result in personal injury.